

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 3

Stromy, kostry, topologické řazení

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 6. října 2025

(Verze dokumentu: 14. 12. 2025 19:21)

Charakterizace stromů

- Kombinace acykličnosti a souvislosti dává stromům několik unikátních vlastností a to umožňuje několik alternativních charakterizací stromů.

Věta (charakterizace stromů)

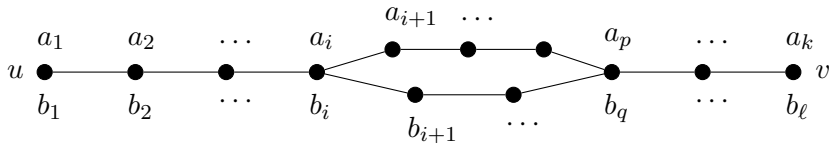
Nechť $G = (V, E)$ je graf. Pak následující tvrzení jsou ekvivalentní:

- 1 G je strom.
- 2 Pro každé dva vrcholy $u, v \in V$ existuje právě jedna u - v -cesta.
- 3 G je souvislý a vynecháním libovolné hrany vznikne nesouvislý graf.
- 4 G je souvislý a $|E| = |V| - 1$.

① G je strom $\Rightarrow$ ② $\forall 2$ vrcholy $u, v \in V \exists$ právě jedna $u-v$ -cesta.

Důkaz:

- Strom je souvislý, proto $\forall x, y \in V \exists x-y$ -cesta.
- Pro spor předpokládejme, že existují dva vrcholy u a v , mezi kterými existují aspoň dvě různé cesty $P_1 = a_1, \dots, a_k$ a $P_2 = b_1, \dots, b_\ell$ (tedy $a_1 = b_1 = u$ a $a_k = b_\ell = v$).
- Nechť $i \geq 1$ je nejmenší index takový, že $a_i = b_i$ a $a_{i+1} \neq b_{i+1}$.
- Označme $p, q > i$ takovou dvojici indexů, že $a_p = b_q$ a $\{a_{i+1}, \dots, a_{p-1}\} \cap \{b_{i+1}, \dots, b_{q-1}\} = \emptyset$.
- Pak ale vrcholy $\{a_i, a_{i+1}, \dots, a_{p-1}, a_p\}$ a $\{b_{i+1}, \dots, b_{q-1}\}$ dohromady tvoří v G kružnici – spor s předpokladem ①. □



② $\forall 2$ vrcholy $u, v \in V \exists$ právě jedna u - v -cesta $\Rightarrow$ ① G je strom.

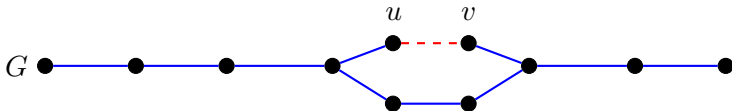
Důkaz:

- Z předpokladu vyplývá, že G je souvislý.
- Pokud by v G byla kružnice, tak by mezi nějakými dvěma jejími vrcholy existovaly alespoň dvě různé cesty, což je spor s předpokladem.
- G je tedy strom. □

❶ G je strom $\Rightarrow$ ❸ G je souvislý a vynecháním libovolné hrany vznikne nesouvislý graf.

Důkaz:

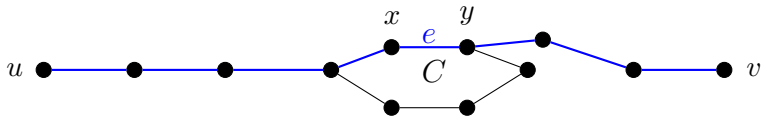
- Souvislost je splněna triviálně.
- Předpokládejme pro spor, že existuje hrana $e = \{u, v\} \in E$ taková, že graf $G - e = (V, E \setminus \{e\})$ je souvislý.
- Pak v $G - e$ musí existovat u - v -cesta, která se ale po přidání hrany e stane v grafu G kružnicí, což je spor s předpokladem, že G je strom. □



③ G je souvislý a vynecháním libovolné hrany vznikne nesouvislý graf
 $\Rightarrow$ ① G je strom.

Důkaz:

- Souvislost je splněna, stačí ukázat neexistenci kružnic v G .
- Předpokládejme pro spor, že (1) vynecháním lib. hrany v G vznikne nesouvislý graf a současně že (2) G obsahuje kružnici C .
- Vyjmeme-li z G ale libovolnou hranu $e = \{x, y\} \in C$, pak se G nerozpadne na dvě komponenty, protože jakékoliv cestě mezi libovolnými 2 vrcholy u a v , která před tím obsahovala e , poskytne C alternativní spojení.
- Pro hrany kružnice C tudíž předpoklad (1) neplatí – dostali jsme spor. □



① G je strom $\Rightarrow$ ④ G je souvislý a $|V| = |E| + 1$.

Důkaz:

- Souvislost je splněna triviálně.
- Pro důkaz $|V| = |E| + 1$ použijeme indukci podle $|V|$.
- Pro $|V| = 1$ tvrzení platí triviálně. Předpokládejme $|V| > 1$.
- Dle Věty o existenci listů existuje v stromu G list v .
- Graf $G' = G - v$ je dle Věty o trhání listů opět strom.
- Dle indukčního předpokladu tedy platí $|V(G')| = |E(G')| + 1$.
- Vratíme-li list v do grafu G , zvýšíme počet vrcholů i počet hran o jedna. Proto
$$|V| = |V(G')| + 1 = (|E(G')| + 1) + 1 = |E| + 1.$$



4 G je souvislý a $|V| = |E| + 1 \Rightarrow$ 1 G je strom.

Důkaz:

- Indukcí podle $|V|$. Pro $|V| = 1$ tvrzení triviálně platí.
- Ukážeme nejdříve, že z předpokladu plyne, že v G existuje list.
 - * Dle předpokladu platí $\sum_{u \in V} \deg_G(u) = 2|E| = 2|V| - 2$.
 - * Kdyby všechny stupně v G byly aspoň 2, pak $\sum_{u \in V} \deg_G(u) \geq 2|V|$.
 - * Tedy nějaký vrchol musí mít stupeň nejvýše 1. Označme ho v .
 - * Protože G je souvislý, vrchol v nemůže mít stupeň 0.
 - * Vrchol v je tedy list.
- Graf $G' = G - v$ je souvislý a splňuje $|V(G')| = |E(G')| + 1$, neboť jsme z G odebrali 1 vrchol a 1 hranu.
- G' je tedy dle indukčního předpokladu strom.
- Dle Věty o trhání listů je tedy G také strom.



Kostru grafu

Motivace:

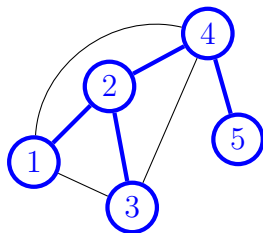
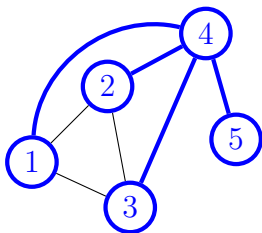
- Uvažujme topologii počítačové sítě popsanou jako souvislý graf, jehož hrany představují komunikační kabely mezi směrovači.
- Ke každému směrovači mohou být připojeny koncové počítače.
- Chceme **minimalizovat počet** komunikačních kabelů tak, aby počítačová síť zůstala funkční, tedy aby libovolný počítač mohl komunikovat s libovolným jiným počítačem díky tomu, že lze postavit mezi nimi komunikační propojení složené z 1 či více směrovačů.
- Problém, které konkrétní kabely zvolit pro toto minimální řešení, je problém konstrukce **kostry grafu** (angl. *spanning tree*) sítě.
- Pokud navíc požadujeme, aby byl **minimální součet délek použitých kabelů**, dostaneme problém **minimální kostry**.

Kostra grafu

Definice

Nechť $G = (V, E)$ je souvislý graf. Podgraf K grafu G nazveme **kostrou** grafu G , pokud $V(K) = V$ a K je strom.

- Nesouvislé grafy nemají kostru, kostru má každá souvislá komponenta.
- Kostra souvislého grafu je tedy souvislý podgraf nad stejnou množinou vrcholů s nejmenším počtem hran.
- Souvislý graf s kružnicemi má více různých koster, např.



Jak najít kostru souvislého grafu?

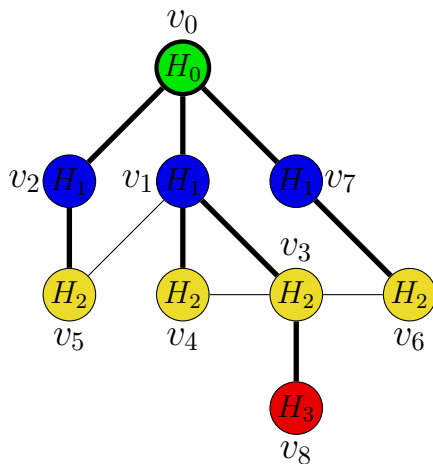
- Tato úloha se může řešit stejně efektivně jak prohledáváním grafu do hloubky tak prohledáváním do šířky.
- Ukážeme si jednoduchou modifikaci algoritmu **BFS**.
- Předpokládejme, že máme ověřeno, že vstupní graf je souvislý (minulá přednáška).

Vstup: Souvislý graf G

Výstup: Seznam hran nějaké kostry grafu G

Jak je třeba BFS modifikovat?

Pro odpověď si stačí připomenout hladiny v **BFS**:



- Hrany do předchůdců tvoří kostru.

Vlastnosti BFS pro hledání kostry

Věta (o správnosti algoritmu `BFS_kostru`)

Nechť $G = (V, E)$ je souvislý graf.

Potom hrany do předchůdců v `BFS` tvoří nějakou kostru grafu G .

Důkaz.

- Označme H graf na množině vrcholů V s hranami do předchůdců (podle pole vyplněného algoritmem `BFS`).
- Víme, že každý vrchol kromě počátečního vrcholu má právě jednoho předchůdce. Tedy H má právě $n - 1$ hran.
- Protože po hranách do předchůdců je možné se z každého vrcholu dostat do s (tedy pro každý vrchol v existuje v H sled z v do s) a protože složením sledů vznikne sled, je graf H souvislý. □

Motivační úloha: Nástroj „make“

Abychom se dostali k jádru problému, odstraníme ze vstupu všechny nepodstatné informace. Výsledek zakreslíme do tzv. **orientovaného grafu**:

```
all: hello
```

```
hello: hello.o  
      gcc hello.o -o hello
```

```
fact.o: fact.c lib.o  
       gcc -c fact.c
```

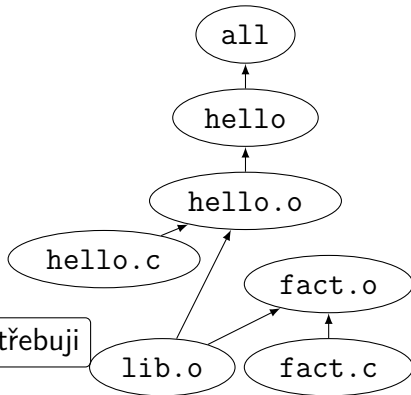
```
hello.o: hello.c lib.o
```

```
gcc -c hello.c lib.o
```

co chci udělat

co k tomu potřebuji

jak to udělat



Formulace úlohy „make“ v řeči teorie grafů

- Vrcholy jsou jednotlivé moduly.
- Pokud má modul y v seznamu prerekvizit modul x , povede orientovaná hrana z x do y (budeme ji značit (x, y)).
- Cílem je **najít pořadí** modulů (čili moduly očíslovat) $m_1, m_2, \dots, m_n$ tak, aby všechny orientované hrany vedly pouze od **nižších** k **vyšším** indexům.
- Tomuto pořadí se říká **topologické uspořádání vrcholů** orientovaného grafu.
- Jinými slovy, je-li $m_1, m_2, \dots, m_n$ topologické uspořádání vrcholů, pak pro každou orientovanou hranu (m_i, m_j) platí $i < j$ (čili m_j je vpravo od m_i neboli všechny šipky směřují zleva doprava).

Topologické uspořádání grafu pořádně

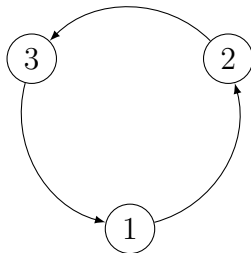
Definice

Topologické uspořádání (TU) orientovaného grafu $G = (V, E)$ je takové seřazení vrcholů $V = \{v_1, \dots, v_n\}$, že pro každou orientovanou hranu $(v_i, v_j) \in E$ platí $i < j$.

- Jinými slovy, pokud vrcholy zakreslíme horizontálně v pořadí topologického uspořádání, budou všechny orientované hrany směřovat zleva doprava.
- Topologické uspořádání se používá především pro plánování pořadí provedení navzájem závislých úloh/výpočtů.

Cyklické závislosti

Získaný orientovaný graf však může obsahovat **cyklus** (orientovanou kružnici).



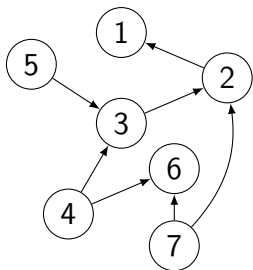
Pozorování

Pokud graf obsahuje cyklus, nelze tuto úlohu zjevně vyřešit.

Definice (Orientovaná kružnice)

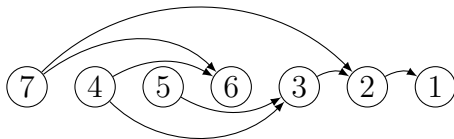
Nechť $n \geq 1$. Orientovaná kružnice délky n (s n vrcholy) je orient. graf $(\{1, \dots, n\}, \{(i, i+1) \mid i \in \{1, \dots, n-1\}\} \cup \{(n, 1)\})$.

Příklad topologického uspořádání grafu



(a) Graf G

7, 4, 5, 6, 3, 2, 1



(b) Příklad TU grafu G

Definice

Orientovaný graf G nazveme **acyklický**, pokud neobsahuje jako podgraf orientovanou kružnici.

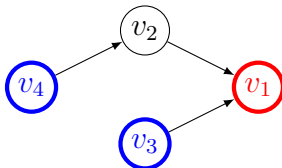
- Anglický termín je *Directed Acyclic Graph* (DAG).
- Zjevně G má více topologických uspořádání.

Úloha „make“ – zdroj a stok

- Předpokládejme nejprve, že vstup je korektní a neobsahuje cyklické závislosti (je to orientovaný **acyklický** graf).

Definice

- **Zdroj** je takový vrchol orientovaného grafu, do kterého nevede žádná hrana.
- **Stok** je takový vrchol orientovaného grafu, ze kterého nevede žádná hrana.



Existence zdroje a stoku v orient. acyklickém grafu

Věta (o existenci zdroje a stoku)

Nechť $G = (V, E)$ je orientovaný acyklický graf. Potom G obsahuje aspoň jeden zdroj a aspoň jeden stok.

Důkaz.

- Sporem pro existenci zdroje (pro stok analogické).
- Předpokládejme, že v G neexistuje zdroj.
- Zvolme libovolný vrchol $v_1 \in V$. Do něj, dle předchozího předpokladu, vede alespoň jedna orientovaná hrana z nějakého vrcholu v_2 .
- Do v_2 také musí vést orientovaná hrana z nějakého vrcholu v_3 , a tak dále.
- Nejpozději po n krocích se však musí nějaký vrchol zopakovat.
- To ale znamená existenci cyklu v G , čímž dostáváme spor. $\square$

Algoritmus TopSort

Existence zdroje v orientovaném acyklickém grafu vede přímočaře na algoritmus konstrukce topologického uspořádání takového grafu.

Vstup: Orientovaný graf G .

Výstup: Někaké topologické uspořádání G , pokud byl acyklický, a detekce cykličnosti v opačném případě.

Idea: Průběžně

- počítej vstupní stupně vrcholů (tj. počet hran, ve kterých je vrchol následníkem),
- zdroje zařazuj do fronty,
- odebírej vrcholy z fronty, zařazuj je do výstupního pořadí a odebírej je z grafu a to znamená, že jejich následníkům snižuj vstupní stupně.
- Pokud po vyprázdnění fronty zbyly v grafu nějaké vrcholy, byl v něm orientovaný cyklus.

Algoritmus TopSort(orientovaný G)

- (1) Q je prázdná fronta
- (2) $\delta[]$ = pole vstupních stupňů vrcholů G ,
na počátku vynulované
- (3) Pro každou hranu $(u, v) \in E(G)$:
- (4) $\delta[v]++$
- (5) Vlož do Q všechny vrcholy z s $\delta[z] = 0$
- (6) Dokud fronta Q není prázdná:
- (7) Odeber prvek z ze začátku fronty Q
- (8) Vypiš z
- (9) Pro každou hranu (z, w) vedoucí ze z :
- (10) $\delta[w]--$
- (11) Pokud $\delta[w] = 0$: zařaď w do Q
- (12) Pokud nebyly zpracovány všechny vrcholy:
- (13) graf G obsahuje orientovaný cyklus

Správnost algoritmu TopSort I

Věta (o správnosti algoritmu TopSort)

Algoritmus **TopSort**(G) spuštěný na orientovaný graf G doběhne v konečném čase a buď vygeneruje korektní topologické uspořádání acyklického grafu nebo detekuje existenci orientovaného cyklu.

Důkaz.

- Předpokládejme, že G je orientovaný graf a předpokládejme, že na počátku v kroku (5) existuje aspoň jeden zdroj z a že všechny zdroje zařadíme do fronty Q .
- Vnitřek cyklu (7)-(11) realizuje odebrání 1 zdroje z fronty Q , jeho vyjmutí z grafu G , jeho vložení do výsledného uspořádání a identifikaci případných nových zdrojů a jejich zařazení do Q .
- Existence hrany (v_i, v_j) znemožňuje vložit vrchol v_j do fronty Q před vrcholem v_i , protože v_j by v takovém okamžiku mělo $\delta[v_j] \geq 1$ a podmínkou vložení do Q je $\delta[v_j] = 0$.
- ...

Správnost algoritmu TopSort II

Věta (o správnosti algoritmu TopSort)

Algoritmus **TopSort**(G) spuštěný na orientovaný graf G doběhne v konečném čase a buď vygeneruje korektní topologické uspořádání acyklického grafu nebo detekuje existenci orientovaného cyklu.

Pokračování důkazu.

- Pokud v okamžiku vyprázdnění fronty jsou zpracovány všechny vrcholy grafu G , našli jsme TU orientovaného acyklického grafu.
- Pokud je fronta prázdná a přesto zbývají nějaké nezpracované vrcholy ve zbytku G , pak ve zbytku G nejsou žádné zdroje, čili každý vrchol má vstupní hranu/-y a musí existovat orientovaný cyklus, plus případně z něj dostupné podgrafy. Vrcholy tohoto zbytkového grafu nelze topologicky uspořádat.
- Algoritmus tedy správně detekuje existenci cyklu a skončí v konečném čase.
- V obou případech algoritmus pracuje korektně a zastaví se. □

Analýza složitosti TopSortu

Věta (o složitosti algoritmu TopSort)

Algoritmus **TopSort**(G), kde $G = (V, E)$ je orientovaný graf reprezentovaný pomocí pole následníků, má časovou i paměťovou složitost $O(|V| + |E|)$.

Důkaz.

- Výpočet pole vstupních stupňů na řádcích (3),(4) trvá $\Theta(|E|)$.
- Počáteční vložení zdrojů do Q na řádku (5) trvá $O(|V|)$.
- Každý vrchol je vložen do Q nejvýše 1x, proto vybírání z Q a výpis na výstup na řádcích (6)–(8) trvá $O(|V|)$.
- Každá hrana může být odebrána z grafu nejvýše 1x, proto operace na řádcích (9)–(11) trvají $O(|E|)$.
- Celková časová složitost je tedy $O(|V| + |E|)$.
- Graf G , pole $\delta[]$, fronta Q , to vše zabere $O(|V| + |E|)$ paměti.



K zamyšlení

- Jak vypadají orientované grafy s právě dvěma topologickými uspořádáními?
- Jak vypadají orientované grafy s právě třemi topologickými uspořádáními?
- Fungoval by algoritmus **TopSort** i s jinou strukturou než frontou?
- Co třeba se zásobníkem?
- Navrhněte, jak modifikovat algoritmus **DFS** pro orientované grafy, aby vytvořil TU grafu.
 - ▶ Náповěda 1: Vezmu-li orientovaný acyklický graf a nějaké jeho TU, jakou vlastnost musí mít poslední vrchol v TU?
 - ▶ Náповěda 2: Připustme, že TU lze konstruovat odzadu.

Řazení – motivační problém

Zpracování úkolů podle priorit

- Představme si manažera, který dostává postupně úkoly spolu s jejich číselnou prioritou (čím menší číslo, tím důležitější úkol).
- Jak si má efektivně zorganizovat zpracování úkolů tak, aby uměl rychle zařazovat nové úkoly a zpracovávat nejprioritnější?

Dvě varianty:

- **offline**: Nejdřív přijme všechny úkoly a pak je zpracuje najednou v pořadí priorit.
- **online**: Úkoly chodí průběžně během zpracovávání předchozích.

Offline verze

- Nejdřív přijdou všechny úkoly, pak jsou zpracovány podle priorit.
- To vyžaduje vzestupné **seřazení** úkolů podle priorit!

Definice (Problém řazení)

- **Vstup:** Číslo n a posloupnost čísel $A = a_1, a_2, \dots, a_n$.
- **Výstup:** Taková permutace $A' = a'_1, a'_2, \dots, a'_n$ vstupní posloupnosti A , že platí $a'_1 \leq a'_2 \leq \dots \leq a'_n$.
- Příklad: $n = 6$, $A = 23, 7, 19, 21, 5, 12$, $A' = 5, 7, 12, 19, 21, 23$.
- Obecně: ze vstupní posloupnosti vytvořit posloupnost hodnot v předdefinovaném pořadí.
- Fundamentální problém v informatice.
- Existuje řada přístupů a algoritmů.
- Implicitně budeme uvažovat algoritmy řadicí **vzestupně**.

Taxonomie algoritmů pro řazení

① Základní operace:

- ① Řazení založené na pouze binární operaci **porovnání-a-prohození** (Compare-And-Exchange).
- ② Řazení založené na jiné operaci.

② Paměťová náročnost:

- ① **In-place** algoritmy (potřebují $n + \text{polylog}(n)$ paměti).
- ② **Out-of-place** algoritmy.

③ Stabilita:

- ① **Stabilní** algoritmy (stejně velké prvky se nikdy neprohodí).
- ② **Nestabilní** algoritmy (stejně velké prvky se mohou prohodit).

④ Citlivost na hodnoty prvků vstupní posloupnosti:

- ① **Datově citlivé** algoritmy (časová složitost závisí na hodnotách prvků).
- ② **Datově necitlivé** algoritmy (časová složitost nezávisí na hodnotách prvků).

SelectSort a InsertSort

- Vstupní posloupnost se rozdělí na **levou seřazenou** a **pravou neseřazenou část**.
- Na počátku je levá část prázdná a pravá část je celá vstupní posloupnost.
- Po vložení prvku se hranice mezi seřazenou a neseřazenou částí posune o jednu pozici doprava.

SelectSort

- Z neseřazené pravé části se vybere **minimum** a prohodí se s jejím **prvním** prvkem, tj. přímo za seřazenou část.

InsertSort

- Z neseřazené pravé části se vezme první prvek a vloží se zprava na správnou pozici v levé seřazené části.

SelectSort a InsertSort

Fakt

Algoritmus **SelectSort** seřadí n -prvkové pole v čase $\Theta(n^2)$ a je **in-place, nestabilní, datově necitlivý**.

- **Invariant:** Po i -té iteraci je vlevo seřazená posloupnost délky i a vpravo zbývá $n - i$ neseřazených prvků, které jsou $\geq$ než čísla v levé části.

Fakt

Algoritmus **InsertSort** seřadí n -prvkové pole v čase $O(n^2)$ a je **in-place, stabilní, datově citlivý**.

- **Invariant:** Po i -té iteraci je vlevo seřazená posloupnost délky $i + 1$ a vpravo zbývá $n - 1 - i$ neseřazených prvků.

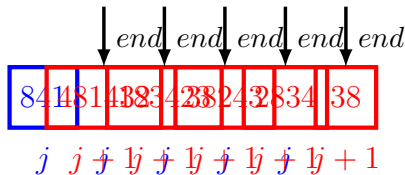
Algoritmus BubbleSort

- Řazení **probubláváním** větších prvků doprava.
- Postupně zleva doprava se porovnávají dvojice **sousedních** prvků a pokud jsou prvky dvojice v nesprávném pořadí, prohodí se.
- Po prvním průchodu bubliny se největší prvek dostane na poslední místo.
- Celý postup se opakuje nejvýše $(n - 1)$ -krát: pokaždé se seřazená podposloupnost vpravo prodlouží o jednu pozici doleva.
- Pokud se během jednoho průchodu neprohodila žádná dvojice sousedů, už se nikdy žádná neprohodí a řazení skončilo.
- Vhodný algoritmus, pokud je např. vstupní posloupnost z velké části seřazená.

Algoritmus BubbleSort – příklad

Algoritmus BubbleSort

```
(1)  $end := n$   
(2)  $zmena := 1$   
(3) Dokud  $zmena = 1$ :  
(4)    $zmena := 0$   
(5)   Pro  $j := 1 \dots (end - 1)$ :  
(6)     Pokud  $P[j] > P[j + 1]$ :  
(7)       prohod'  $P[j]$  a  $P[j + 1]$   
(8)        $zmena := 1$   
(9)    $end --$ 
```



Korektnost algoritmu BubbleSort

Věta

Algoritmus **BubbleSort** korektně seřadí n -prvkové pole.

Důkaz.

- Korektnost dokážeme indukčním důkazem následujícího invariantu.
- **Invariant:** nejpozději po i -té iteraci cyklu na řádcích (3)–(9) se i největších prvků pole nachází seřazeno na i posledních pozicích zprava.
- IZ: Invariant platí pro $i = 1$. Po 1.iteraci je největší prvek celé posloupnosti zaručeně na poslední pozici.
- IK: Předpokládejme, že Invariant platí pro všechna $j < i$ a tedy $i - 1$ největších prvků se nachází seřazeno na posledních $i - 1$ pozicích vpravo. Provedme i -tou iteraci.
- Tím probublá největší hodnota levého zbytku doprava, levá část se zkrátí o jedna a pravá seřazená část se zvětší o jeden prvek, a invariant platí.



Složitost a vlastnosti algoritmu BubbleSort

Věta

BubbleSort seřadí n -prvkové pole v čase $O(n^2)$ a je **in-place**, **stabilní**, **datově citlivý**.

Důkaz.

- Iterací je nejvýše $n - 1$, i -tá iterace trvá $\Theta(n - i)$ kroků, což dává časovou složitost $O(n^2)$.
- Kromě vstupního pole potřebuje pouze $O(1)$ paměti.
- Je stabilní, neboť podmínka na řádku (6) zabrání prohození stejných prvků.
- Je datově citlivý, neboť končí, jakmile nedošlo během některé iterace ke změně. V nejlepším případě již seřazeného pole má složitost $\Theta(n)$.



Algoritmy řazení: Shrnutí

- Ukázali jsme si tři základní příklady tzv. **kvadratických** řadicích algoritmů, tedy pracujících v čase $O(n^2)$.
- Jsou velmi jednoduché, intuitivní a pro menší vstupní posloupnosti dat vyhovující a prakticky použitelné.
- Dva z nich jsou datově citlivé a pro vstupy částečně předseřazené dosahují časů blízkých lineárním.
- Existují ale asymptoticky rychlejší algoritmy v nejhorším případě, se kterými se začneme seznamovat příště.

Co bychom po dnešku měli znát

- Charakterizace stromů.
- Kostra grafu.
- Topologické uspořádání.
- Věta o existenci zdroje.
- Algoritmus nalezení nějaké kostry grafu.
- Algoritmus konstrukce nějakého topologického uspořádání orientovaného grafu.
- Problém řazení a kvadratické algoritmy.